Bitcoin Mining

Hasitha Peter z5518219
Qiyue Wang (Martin) z5484179
Pyae Sone Oo (Morris) z5271704
Timothy Lam z5481745
Dylan Loh z5432665

Problem Outline

- Bitcoin mining is a brute-force search: hash an 80-byte block header, check the result, bump the nonce and try again.
- The hash is SHA-256d which is SHA-256 running twice.
- A nonce “wins” if the hash lands below a target. Even at the easiest difficulty in Bitcoin’s history, that is around four billion attempts on average.
- Each attempt is independent and SHA-256 is nothing but 32-bit adds, rotates and bitwise logic. That is exactly what FPGA fabric is good at.
- FPGAs mined real Bitcoin until ASICs pushed them out in 2013. We are redoing it with modern HLS tools on the KV260.

Project Aims

- Build a SHA-256d mining core in Vitis HLS and run it on the KV260 board
- Keep the whole nonce search inside the programmable logic. The ARM cores only hand out work and verify answers.
- Measure the hash rate and efficiency (energy per hash) against a tuned software miner on a modern CPU and on the KV260’s ARM cores.
- Working target: at least 20x the software hash rate (to be firmed up after profiling)

SHA-256 algorithm

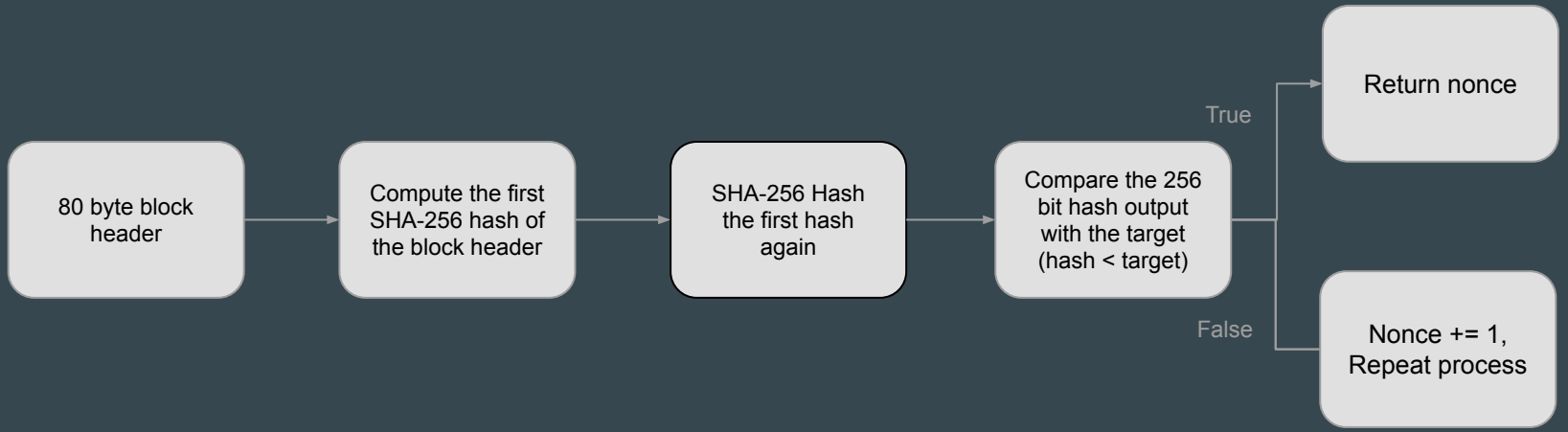
- SHA-256 is a hash function which converts an input of any size into a fixed 256 bit hash output
- A small change in the input produces a completely different hash

Step by step for SHA-256 Process

1	**Input Preparation** Read the 80-byte block header and pad the message so its length becomes a multiple of **512 bits**.
2	**Parsing the message** The padded message gets divided into 512-bit blocks
3	**Initialise hash variables** Initialised hash values for computation
4	**Message Schedule** Expand each block from 512-bits into 64 words (32 bits each**)** used during hashing.
5	**Compression Function** Perform 64 rounds of logical operations (XOR, AND, rotations, additions) for each block
6	**Generate Hash** Combine the updated hash values to produce the final 256-bit digest(64 hexadecimal characters)

BITCOIN Mining Process

- In the BITCOIN mining process, the 80-byte block header is repeatedly hashed using double SHA-256. A valid nonce is found when the resulting hash is less than the target difficulty



Investigation so far

01 . BITCOIN MINING & SHA-256

- Reviewed Bitcoin and its use of SHA-256 to ensure that its blockchain is kept valid without a trusted third party.
- Analyzed SHA-256 and its 6-stage process where an initial input is repeatedly mixed until computationally irreversible.

03 . HLS OPTIMIZATIONS

Numerous optimizations exist within message scheduling and compression. These mixing operations are massively parallelizable on FPGA architecture, complemented by macro dataflow parallelization for maximum hashrate. [3]

02 . BASELINE BENCHMARKS

Repeatedly hashing an 80-byte header to reach target difficulty. Measured by hashrate and efficiency.

Arch	Hashrate	Efficiency
Kria KV260 PS (Cortex-A53)[1]	75.4 Kh/s	-
Modern GPU (RTX 3090) [1]	~0.33-0.41 Gh/s	~0.840 j/Th
ASICs (SealMiner A4) [2]	886 Th/s	9.449 j/Th

04 . PROFILING AND PROBING

Begun our real-world investigation of double-SHA-256 on the Kria KV260 platform, referencing a standard baseline C++ implementation.

System-Glitch: SHA256

Sources: [1] [2] [3]
Note 1: Baseline unoptimised single-core implementation

Project plan & Milestones

Phase 1: Software Baseline (Week 6)

Build and test a CPU-based SHA-256d Bitcoin mining loop on the KV260 ARM processor.

Output: baseline hash rate and correctness tests.

Phase 2: HLS Mining Core (Week 7)

Move the SHA-256d nonce-search kernel into HLS and verify against the software version.

Output: working synthesised accelerator core.

Phase 3: KV260 Integration, (Week 8 - 9)

Connect the HLS core to the ARM host program using AXI and run the full system on hardware.

Output: first end-to-end FPGA mining test.

Phase 4: Optimisation & Final Evaluation, (Week 9 - 10)

Apply midstate reuse, pipelining, loop unrolling or parallel nonce lanes. Compare software and hardware performance.

Output: speedup, resource usage, power results, demo and final report.

Risks & Contingencies

Risk 1: FPGA resource utilisation

- High-performance double SHA-256 implementations require approximately 46,000 LUTs and over 52,000 flip-flops. Although these requirements appear to fit within the KV260's available resources, resource constraints may limit future expansion or additional hashing pipelines.
- Contingency: Start with a single working double SHA-256 pipeline and check post-synthesis/post-implementation utilisation before adding extra features.

Risk 2: Scalability

- Increasing throughput by replicating SHA-256 pipelines may eventually exceed available programmable logic or introduce routing congestion that prevents timing closure.
- Contingency: Scale incrementally: implement one pipeline first, then add pipelines one at a time while checking timing, routing congestion, and resource use after each build.

Risk 3: **Failed baseline**

- The base implementation for the bitcoin miner may fail, leading to an unoptimized and faulty project.
- Contingency: Prioritise implementing baseline before the optimizations.

Roles

Tasks	Description	Assigned Member
Software Baseline	Develops the C/C++ SHA-256d miner, prepares test cases, and checks hardware results against the software output.	Hasitha
HLS SHA-256 Core	Implements the SHA-256 compression function in HLS, runs C simulation/co-simulation, and records synthesis results.	Dylan
Mining Kernel & Control Logic	Implements the SHA-256 compression function in HLS, runs C simulation/co-simulation, and records synthesis results.	Martin
KV260 Integration & Host Program	Builds the full nonce-search accelerator around the SHA-256 core, including nonce generation, target comparison, and result output.	Morris
Optimisation, Benchmarking & Documentation	Tests pipelining, unrolling, midstate reuse or parallel lanes, collects performance/resource/power results, and maintains slides, report and project updates.	Timothy